

What Happens Inside Kubernetes After `kubectl apply`

Command

```
kubectl apply -f busybox-pod.yml
```

One-line explanation

This command tells Kubernetes to create or update the resources defined in the YAML file so the cluster matches the desired state.

Step-by-step flow inside Kubernetes

1. *`kubectl` reads the YAML file*

Your local machine reads `busybox-pod.yml` and understands what resource you want to create.

Example:

- `kind: Pod`
- `metadata.name: busybox-pod`

At this point, `kubectl` knows you want a Pod named `busybox-pod`.

2. *`kubectl` sends the request to the API Server*

`kubectl` does not create the Pod directly.

It sends the resource definition to the **Kubernetes API Server**, which is the main entry point for all cluster operations.

3. *The API Server validates the request*

The API Server checks things like:

- Is the YAML valid?
- Is `Pod` a valid Kubernetes resource?
- Is the schema correct?
- Does the user have permission to create it?

If something is wrong, Kubernetes returns an error here.

If everything is correct, the request moves forward.

4. *The desired state is stored in `etcd`*

After validation, Kubernetes stores the object in **etcd**, which is the cluster's key-value database.

This means Kubernetes records the desired state, for example:

> I want a Pod named `busybox-pod` running.

5. The Scheduler selects a node

If the Pod is not yet assigned to a node, the **Scheduler** looks at the available nodes and decides where to place it.

It considers things like:

- Resource availability
- Node constraints
- Taints and tolerations
- Affinity and anti-affinity rules

Then it assigns the Pod to a suitable node.

6. Kubelet notices the Pod assignment

On the selected node, the **kubelet** watches for Pods assigned to that node. When it sees `busybox-pod`, it begins the work needed to run it.

7. The container runtime pulls the image and starts the container

The kubelet asks the container runtime (such as **containerd** or **Docker**, depending on setup) to:

- Pull the image: `busybox:latest`
- Create the container
- Run the command inside it

For your example, the command is:

```
echo Hello from BusyBox && sleep 3600
```

So the container:

1. Prints `Hello from BusyBox`
2. Sleeps for 3600 seconds

8. Pod networking and storage are prepared

Before the container is fully running, Kubernetes also sets up:

- The Pod IP address
- Network namespace
- Mounted volumes, if any are defined

This makes the Pod ready to communicate inside the cluster.

9. The Pod status is updated

Once the container starts successfully, the kubelet reports the status back to the API Server.

You can then check it with:

```
kubectl get pods
```

Possible statuses include:

- `Pending`
- `ContainerCreating`
- `Running`
- `CrashLoopBackOff`
- `Completed`

10. Kubernetes keeps reconciling desired state and actual state

Kubernetes continuously checks whether the actual state matches the desired state.

If something goes wrong, Kubernetes components work to bring the cluster back to the expected state.

For a plain Pod, recovery is limited compared with a Deployment, but the reconciliation idea is the same.

Visual flow

```
kubectl
↓
API Server
↓
etcd (stores desired state)
↓
Scheduler (chooses node)
↓
Kubelet (on selected node)
↓
Container Runtime
↓
Pod Running
```

Interview-ready answer

When you run `kubectl apply -f busybox-pod.yml`, `kubectl` sends the YAML definition to the API Server. The API Server validates it and stores the desired state in `etcd`. The Scheduler chooses a node, the kubelet on that node asks the container runtime to pull the image and start the container, and then Kubernetes updates the Pod status to show whether it is running successfully.

Key concept to remember

Kubernetes always works on:

****Desired State vs Current State****

You declare:

> I want this Pod to exist.

Kubernetes works continuously to make reality match that declaration.